

다양한 서버와 GitHub 시작하기

요청이 처리되는 구조부터

코드를 기록하고 함께 배포하는 과정까지



초급프로젝트 · WEEK 02

오늘의 강의 구성

서버에서 시작해
코드로 기록하고 공유합니다.

01 서버가 일하는 방식

요청과 응답 · 웹 화면 · HTTP

02 Git으로 변경 기록하기

add · commit · diff · restore

03 브랜치와 변경 이력

switch · merge · 충돌 해결

04 GitHub에서 코드 공유하기

clone · push · pull · Pull Request

05 코드를 웹페이지로 배포하기

index.html · GitHub Pages

서버는 요청에 응답하는 역할입니다



주소 입력부터 화면 표시까지



05 API가 JSON 응답을 보내면 브라우저가 공지 목록을 갱신합니다.

DNS를 터미널에서 조회하기

실행 예시

```
nslookup example.com
```

```
# 출력의 주요 항목
```

```
Name: example.com
```

```
Address: <조회된 IP 주소>
```

```
# 같은 이름에 여러 주소가 나올 수 있음
```

```
# DNS 캐시에 따라 응답이 달라질 수 있음
```

이름에서 주소로

DNS 조회는 도메인에 대응하는 IP 주소 등의 정보를 찾습니다.

웹 요청과 구분

nslookup은 HTML을 내려받지 않습니다. 주소를 얻은 뒤 HTTP 요청을 보냅니다.

실행 환경

macOS·Windows에서 사용할 수 있습니다.
Linux는 배포판에 따라 DNS 도구 설치가 필요합니다.

웹 서버와 API 서버의 역할

웹 서버

HTML CSS JS

브라우저에 파일을 전달

예: Nginx, Apache HTTP Server. 정적 파일 제공과 요청 전달을 맡을 수 있습니다.

API / 앱 서버

```
GET /api/notices  
{ "title": "모집 안내" }
```

규칙에 따라 요청을 처리

예: Node.js, Spring Boot, Flask로 만든 앱. 조회·로그인·권한 확인 등을 처리합니다.

서비스가 커지면 역할을 더 나눅니다

파일 / 오브젝트 저장소

.jpg .pdf

이미지와 첨부파일 보관

API는 파일의 주소와 권한을 관리할 수 있습니다.

인증 서버

로그인 확인

세션 / 토큰 발급

사용자가 누구인지 확인

실제 작업의 허용 여부인 인가는 API에서도 검증합니다.

캐시 서버

HIT / MISS

자주 쓰는 값을 빠르게 재사용

캐시가 없으면 원본 조회. 원본 변경 시 오래된 값에 주의합니다.

요청의 앞단에도 서버가 있습니다

01. DNS

도메인 이름을 IP 주소 등으로 연결하는 정보를 제공합니다.

02. 리버스 프록시

클라이언트 앞에서 요청을 받아 내부 서버로 전달합니다. TLS 종료나 부하 분산도 맡을 수 있습니다.

03. CDN

여러 지역의 서버에서 캐시된 콘텐츠를 가까이 전달합니다. 요청 설정에 따라 원본 서버를 조 회합니다.

리버스 프록시가 URL에 따라 요청을 나눕니다

실행 예시

```
server {  
    listen 80;  
    root /srv/club-site;  
  
    location / {  
        try_files $uri $uri/ =404;  
    }  
    location /api/ {  
        proxy_pass http://127.0.0.1:3000;  
    }  
}
```

정적 파일

/index.html 등의 요청은 /srv/club-site에서 파일을 찾아 제공합니다.

API 요청

/api/notices 요청은 내부 3000 포트의 앱 서버로 전달합니다.

예시 범위

Nginx server 블록 예시입니다. HTTP 교육용이며 실제 운영의 TLS 설정은 생략했습니다.

"무슨 일?"과 "어떻게 실행?"은 다른 분류입니다

역할로 분류

웹 / API

인증 / 파일 / 캐시

무슨 요청을 처리하나요?

서비스의 기능과 책임을 나누는 기준입니다.

실행 방식으로 분류

물리 서버 / 가상 머신

컨테이너 / 서버리스

어떤 환경에서 실행하나요?

API 서버를 컨테이너로 실행하거나 서버리스 함수로 구현할 수 있습니다.

정적 웹사이트도 버튼에 반응할 수 있습니다

01. 정적 제공

미리 준비한 HTML·CSS·JS를 전달합니다. 브라우저의 **JavaScript**로 상호작용할 수 있습니다.

02. 동적 생성

서버가 요청과 데이터에 따라 HTML 또는 JSON 등의 **응답 내용을 생성**합니다.

03. 프로젝트 선택

소개 페이지는 정적 제공으로 시작할 수 있습니다. 로그인·공유 데이터 저장은 **별도 백엔드 기능**이 필요합니다.

CSR과 SSR: HTML을 어디에서 만들까요?

CSR

서버: JS·데이터 제공

브라우저: 화면 구성

브라우저에서 콘텐츠 구성

초기 JS 실행과 데이터 요청 시간이 사용자 경험에 영향을 줍니다.

SSR

서버: HTML 생성

브라우저: 표시·상호작용

서버에서 초기 HTML 구성

초기 콘텐츠를 전달하고 이후 브라우저가 상호작용을 이어갈 수 있습니다.

SPA·MPA와 CSR·SSR은 같은 구분이 아닙니다

01. SPA / MPA

페이지 이동 시 **한 문서 안에서 화면을 전환**하는지, 새 HTML 문서를 받아 이동하는지의 구분입니다.

02. CSR / SSR

콘텐츠의 HTML을 **브라우저에서 구성**하는지, 서버에서 구성하는지의 구분입니다.

03. 함께 사용 가능

SPA 방식의 앱도 초기 화면에 SSR을 사용할 수 있습니다. **항상 빠른 방식은 없고** 요구사항에 맞춰 선택합니다.

localhost는 지금 접속한 내 컴퓨터입니다

실습 코드

```
http://localhost:8000
```

```
# index.html이 있는 폴더에서 실행
```

```
# macOS / Linux (Python 3 필요)
```

```
python3 -m http.server 8000 --bind 127.0.0.1
```

```
# Windows (Python launcher 필요)
```

```
py -m http.server 8000 --bind 127.0.0.1
```

주소 읽기

localhost는 내 컴퓨터, 8000은 서비스가 듣는 포트입니다.

서버 실행

브라우저로 위 주소를 열어 봅니다. 종료는 터미널에서 Ctrl+C입니다.

친구와 공유?

친구의 localhost는 친구 컴퓨터입니다. 공개 URL을 얻으려면 배포가 필요합니다.

HTTP 요청을 코드로 보면

실행 예시

```
curl -i http://localhost:8000/
```

응답 예시

```
HTTP/1.0 200 OK
```

```
Content-Type: text/html
```

```
<!doctype html>
```

```
<html lang="ko">
```

```
    <h1>동아리 모집</h1>
```

```
</html>
```

curl -i

앞 장의 Python 서버 실행 중 사용합니다. 헤더와 본문을 함께 출력합니다.

200 OK

요청에 성공했다는 HTTP 상태입니다. 화면 그림 대신 HTML 텍스트를 받습니다.

브라우저의 역할

HTML을 화면으로 구성하고 CSS·JS 요청을 이어갑니다. Windows에서는 curl.exe를 쓸 수 있습니다.

성공한 요청과 없는 파일의 응답

실행 예시

```
# Python 로컬 서버 실행 중
curl -I http://localhost:8000/index.html
# HTTP/1.0 200 OK
# Content-type: text/html

curl -i http://localhost:8000/missing.html
# HTTP/1.0 404 File not found

# -I: HEAD 요청, 응답 헤더만 표시
# -i: 응답 헤더와 본문을 함께 표시
```

200과 404

요청한 파일이 있는 경우와 없는 경우를 비교합니다.
서버가 실행 중이어야 합니다.

요청 자체가 실패하면

Connection refused는 HTTP 404와 다릅니다.
서버 실행과 포트부터 살펴봅니다.

Network와 연결

브라우저 개발자 도구의 Network 탭에서도 요청
URL과 응답 상태를 볼 수 있습니다.

Git은 기록 도구, GitHub는 협업 서비스입니다

Git

LOCAL

내 컴퓨터에서 이력 관리

인터넷 없이도 커밋·비교·브랜치 작업을 할 수 있습니다.

GitHub

REMOTE

원격 저장소와 협업 공간

저장소 공유, Issue, Pull Request, 코드 리뷰 등을 제공합니다.

저장소는 프로젝트 파일과 Git 기록을 함께 관리합니다

기본 실습

```
club-site/  
├─ index.html    # 웹페이지  
├─ README.md     # 프로젝트 설명  
└─ .git/         # Git이 관리하는 기록
```

현재 폴더를 Git 저장소로 만들기

```
git init -b main
```

지금 파일 상태 보기

```
git status
```

저장소 = 프로젝트의 기록 공간

프로젝트 파일과 변경 이력을 함께 관리합니다. 이 수업에서는 club-site 폴더 하나를 사용합니다.

.git은 Git이 관리

숨김 폴더라 탐색기에서 보이지 않을 수 있습니다. 안에 있는 파일을 직접 수정할 필요는 없습니다.

시작할 때 한 번 init

새 로컬 프로젝트에서 실행합니다. GitHub 저장소를 clone했다면 init을 다시 할 필요가 없습니다.

예제 프로젝트: 동아리 소개 페이지

실행 예시

```
club-site/  
├─ index.html  
├─ README.md  
└─ .gitignore
```

index.html의 첫 내용

```
<h1>우리 동아리</h1>
```

작업 흐름

소개 수정 → 기록 → 브랜치 → 공유

하나의 예제로 연결

페이지 제목과 README를 수정하면서 Git의 파일·이력 변화를 봅니다.

명령 실행 환경

macOS·Linux 터미널 또는 Windows Git Bash
기준입니다. \$ 기호는 입력하지 않습니다.

시연 방식

자동 재생, 이전·다음 명령, 명령 행 클릭으로 원하는
상태를 설명합니다.

명령은 한 줄씩 실행하고 결과를 읽습니다

기본 실습

```
cd club-site
# club-site 폴더 안으로 이동

git status
# 상태만 조회: 파일을 바꾸지 않음

git add README.md
# README.md 내용을 다음 커밋에 준비

git commit -m "docs: add introduction"
# 따옴표 안은 커밋 설명
```

어디에 입력하나요?

터미널 또는 편집기의 터미널에 입력하고 Enter를 누릅니다. Windows 실습은 Git Bash를 사용합니다.

git 다음은 동작 이름

status는 상태 보기, add는 내용 준비, commit은 기록하기입니다. 뒤의 파일명은 대상을 지정합니다.

#으로 시작하는 줄

설명을 위한 주석입니다. 오류가 나오면 다음 줄로 넘어가기 전에 메시지와 현재 폴더를 읽습니다.

Git 설치와 저장소별 사용자 설정

실행 예시

```
git --version
mkdir club-site
cd club-site
git init -b main
git config user.name "본인 이름"
git config user.email "본인 이메일"
git config --get user.name
git config --get user.email
```

init -b main

현재 폴더에 .git 저장소를 만들고 기본 브랜치 이름을 main으로 정합니다.

user.name / email

커밋에 기록할 작성자 정보입니다. GitHub 로그인 정보와는 구분됩니다.

저장소별 설정

--global 없이 실행하면 이 저장소에만 적용됩니다.
Git 2.28 이상을 기준으로 합니다.

명령을 실행하는 폴더가 저장소를 결정합니다

실행 예시

```
pwd
ls
cd club-site
ls -a

git rev-parse --show-toplevel
git status

# 상위 폴더로 이동
cd ..
```

pwd / cd

pwd는 현재 경로를, cd는 이동할 경로를 다룹니다.
Git Bash와 macOS/Linux 기준입니다.

.git 폴더

Git이 저장소 이력과 설정을 관리하는 곳입니다.
소스 폴더와 함께 함부로 지우지 않습니다.

저장소 최상위

rev-parse --show-toplevel은 현재 Git 저장소의
최상위 경로를 출력합니다.

작업 폴더·스테이징·커밋은 서로 다른 상태입니다

Git 개념 · 코드로 읽기

1. 편집기에서 파일 수정 후 저장
→ 작업 폴더가 바뀜
 2. **git add** README.md
→ 지금 내용을 스테이징에 준비
 3. **git commit -m "docs: update intro"**
→ 준비된 내용을 변경 이력에 기록
- # 언제든지 현재 상태 보기
- git status**

작업 폴더

지금 편집 중인 파일 내용입니다. 저장하면 이곳이 바뀝니다.

스테이징 = 인덱스

다음 커밋의 파일 상태를 준비하는 공간입니다.
add는 그 시점의 내용을 복사해 반영합니다.

커밋 = 기록된 스냅샷

commit 후에도 인덱스는 남습니다. HEAD와 같아지므로 “커밋할 변경 없음”으로 보입니다.

git status에서 먼저 읽을 세 가지 항목

기본 실습

git status

Untracked files:

notes.txt

Changes not staged for commit:

modified: README.md

Changes to be committed:

modified: index.html

Untracked files

새로 만든 파일 중 Git이 아직 추적하지 않는 파일입니다. 기록하려면 add합니다.

Changes not staged

기존 파일을 수정했지만 아직 add하지 않은 내용입니다. 지금 commit해도 이 수정은 들어가지 않습니다.

Changes to be committed

다음 기본 commit에 들어갈 변경입니다. 이 예시에서는 index.html의 준비된 변경만 기록됩니다.

파일 생성 → add → 첫 commit

명령 · 클릭하면 해당 단계

01 `echo "# Club" > README.md`

02 `git status --short`

03 `git add README.md`

04 `git commit -m "docs: add README"`

05 `git log --oneline`

작업 폴더
변경 없음

→

스테이징
변경 없음

커밋 이력
a1b2c3d

←

브랜치
HEAD → main

실행 결과 예시 · 출력 일부 생략

a1b2c3d (HEAD -> main) docs: add README

05 HEAD는 현재 작업 위치, main은 이 커밋을 가리키는 브랜치 이름입니다.

새 파일이 추적되고 다시 수정되는 네 단계

명령 · 클릭하면 해당 단계

01 `echo "memo v1" > notes.txt`
`git status`

02 `git add notes.txt`
`git status`

03 `git commit -m "docs: add memo"`
`git status`

04 `echo "memo v2" > notes.txt`
`git status`

작업 폴더
v2

→

스테이징 / 인덱스
v1

←

파일 상태
Modified · _M

최근 커밋 HEAD
v1

실행 결과 예시 · 출력 일부 생략
Changes not staged for commit:
 modified: notes.txt

short: M notes.txt

04 Modified: 작업 폴더만 v2입니다. v2를 다음 커밋에 담으려면 다시 add해야 합니다.

서버 이해 > Git 기록 > 브랜치 > GitHub 협업 > 웹 배포

26 / 60 · MacsLAB

수정된 줄을 diff로 읽기

명령 · 클릭하면 해당 단계

01 `echo "# Club 2026" > README.md`

02 `git diff`

03 `git add README.md`

04 `git diff --staged`

작업 폴더
Club 2026

→

스테이징
Club 2026

HEAD
Club

←

현재 비교
스테이지 ↔ HEAD

실행 결과 예시 · 출력 일부 생략

-# Club

+# Club 2026

04 --staged는 스테이징과 HEAD를 비교합니다. 다음 커밋에 기록할 변경입니다.

diff에서 추가된 줄과 삭제된 줄을 읽습니다

Git 개념 · 코드로 읽기

```
diff --git a/README.md b/README.md
--- a/README.md
+++ b/README.md
@@ -1,3 +1,3 @@
 # Club
-Meeting: Monday
+Meeting: Friday
  Place: Room 201

# 변경 파일명만 보기
git diff --name-only
# 파일별 변경량 요약
git diff --stat
```

a와 b는 비교 양쪽

이전 상태의 README.md와 이후 상태의 README.md를 뜻합니다. 실제 a/ 폴더가 필요한 것은 아닙니다.

-는 삭제, +는 추가

월요일을 금요일로 바꾸면 이전 줄 삭제와 새 줄 추가로 표시됩니다. 공백으로 시작하는 줄은 문맥입니다.

먼저 +와 -에 집중

@@는 변경된 줄의 위치를 알려줍니다. 처음에는 숫자를 외우기보다 실제 코드가 어떻게 바뀌었는지 읽습니다.

add한 뒤 다시 수정하면

명령 · 클릭하면 해당 단계

01 `echo "# Club v1" > README.md`

02 `git add README.md`

03 `echo "# Club v2" > README.md`

04 `git commit -m "docs: title v1"`

05 `git show HEAD:README.md`

작업 폴더
v2

→

스테이징
v1

HEAD
v1 커밋

←

status
M README.md

실행 결과 예시 · 출력 일부 생략

`# Club v1`

05 HEAD의 파일을 읽으면 v1입니다. v2를 기록하려면 다시 add하고 commit합니다.

저장·add·commit은 각각 언제 필요할까요?

Git 개념 · 코드로 읽기

편집기에서 README.md를 수정하고 저장

```
git diff -- README.md
```

스테이징에 아직 없는 수정 보기

```
git add README.md
```

지금 파일 내용을 스테이징에 반영

```
git diff --cached -- README.md
```

커밋에 들어갈 변경 보기

```
git commit -m "docs: add club schedule"
```

스테이징의 변경을 로컬 이력으로 기록

저장은 디스크에 반영

편집기의 저장 버튼은 Git 커밋을 만들지 않습니다.
저장 후에도 변경은 미커밋 상태일 수 있습니다.

add는 커밋 내용 준비

파일을 다시 수정했다면 새 내용을 담기 위해 add를
다시 실행합니다.

commit은 로컬 이력 생성

커밋 이후에도 추가 편집은 가능합니다. GitHub에
보내는 push는 별도의 단계입니다.

짧은 status 출력은 필요한 경우에만 읽습니다

실행 예시

```
git status --short
```

```
# XY 파일명 : X와 Y는 각각 한 칸
```

```
?? new.txt      # 아직 추적하지 않음
```

```
M README.md    # 작업 폴더만 다름
```

```
M README.md    # 인덱스만 HEAD와 다름
```

```
MM README.md   # 두 비교 모두 다름
```

```
A notes.txt    # 새 파일 add 완료
```

```
# 공백 = 해당 비교에서 변경 없음
```

```
# 깨끗한 추적 파일은 줄 자체가 안 나옴
```

```
# 위 줄들은 서로 다른 상태의 예시
```

첫 번째 문자

HEAD ↔ 인덱스 비교입니다. 여기에 A나 M이 있으면 다음 기본 commit에 담길 변경이 있습니다.

두 번째 문자

인덱스 ↔ 작업 폴더 비교입니다. M은 add에 아직 반영하지 않은 수정이 있음을 뜻합니다.

MM은 두 상태가 공존

커밋 v1·인덱스 v2·작업 폴더 v3이면 MM입니다. 지금 commit하면 v2가 기록되고 v3 수정은 남습니다.

파일을 골라서 커밋하기

명령 · 클릭하면 해당 단계

01 `echo "소개 추가" >> README.md`

02 `echo "<h1>Club</h1>" > index.html`

03 `git add README.md`

04 `git commit -m "docs: add intro"`

README.md
기록됨

→

index.html
수정만

스테이징
HEAD와 동일

←

새 커밋
README.md

실행 결과 예시 · 출력 일부 생략

`[main c3d4e5f] docs: add intro`

04 문서 변경만 기록합니다. index.html 수정은 그대로 남습니다.

커밋은 한 가지 변경 이유를 설명할 수 있게 나눕니다

Git 개념 · 코드로 읽기

```
# 일정 안내만 먼저 기록
git add README.md
git commit -m "docs: add meeting schedule"

# 버튼 기능은 별도 기록
git add index.html
git commit -m "feat: add visit counter"

# 두 변경의 목적이 이력에 드러남
git log --oneline -2
```

변경의 목적을 기준으로

문서 수정과 버튼 기능 추가는 서로 다른 이유입니다. 따로 기록하면 리뷰와 취소가 쉬워집니다.

메시지는 구체적으로

update보다 add meeting schedule처럼 무엇이 바뀌었는지 적습니다. 필요하다면 본문에 이유를 씁니다.

접두사는 팀의 약속

docs:·feat:는 흔한 작성 관례입니다. Git이 요구하는 문법은 아니므로 팀 규칙에 맞춥니다.

한 번의 수정을 기록하는 기본 순서

기본 실습

1. 편집기에서 README.md 수정 후 저장

git status

2. 무엇을 바꿨는지 보기

git add README.md

3. 이 파일의 현재 내용을 준비

git commit -m "docs: update introduction"

4. 준비한 내용을 로컬 이력에 기록

git status

5. 남은 변경이 있는지 보기

작게 수정하고 기록

한 번에 문장 하나나 기능 하나를 바꿉니다. 수정 이유를 한 문장으로 설명할 수 있게 합니다.

중간에는 상태를 읽기

add 전에는 미준비 수정, add 후에는 커밋 예정 변경으로 표시됩니다.

clean이면 로컬 기록 완료

작업 폴더에 남은 변경이 없다는 뜻입니다. GitHub에 보내려면 뒤에서 배울 push가 필요합니다.

git log로 내가 남긴 기록을 읽습니다

기본 실습

```
git log --oneline -5
```

```
# 출력 예시: 최신 기록이 위쪽
```

```
# b2c3d4e docs: add meeting schedule
```

```
# a1b2c3d docs: add introduction
```

```
# 가장 최근 커밋의 변경 보기
```

```
git show
```

```
# 긴 화면에서 나가려면 q
```

한 줄에 커밋 하나

왼쪽은 커밋을 구분하는 ID, 오른쪽은 내가 적은 설명입니다. ID를 외울 필요는 없습니다.

무엇을 바꿨는지 읽기

git show는 최근 커밋의 변경 내용을 보여줍니다.
+와 -로 추가·삭제된 줄을 읽습니다.

조회는 파일을 바꾸지 않음

log와 show는 기록을 읽는 명령입니다. 파일 복원과는 다른 동작입니다.

잘못 add했다면 커밋 준비만 취소할 수 있습니다

기본 실습

```
# README.md는 이미 커밋된 파일
# 수정 내용을 add한 상태에서 시작

git restore --staged README.md

# 수정한 파일은 그대로 남아 있음
git status
# Changes not staged for commit:
#   modified: README.md

# 다시 담으려면
git add README.md
```

--staged가 핵심

다음 커밋에 담겠다는 준비를 취소합니다. 편집한 파일 내용을 지우지 않습니다.

커밋 전 선택을 바꾸기

이번에 담을 파일을 잘못 골랐을 때 사용합니다. 이미 만든 커밋을 취소하는 명령은 아닙니다.

파일 내용 삭제와 구분

옵션 없는 git restore는 작업 내용을 덮어쓸 수 있습니다. 여기서는 --staged 형태만 연습합니다.

.gitignore로 추적 대상을 관리하기

실습 코드

```
# .gitignore 예시
.env
node_modules/
.DS_Store

# 커밋 전 확인
git status
git diff --staged
```

.gitignore

아직 추적하지 않는 파일을 무시하는 규칙입니다.
이미 추적한 파일에는 소급 적용되지 않습니다.

공유할 것

소스 코드, README, 재현에 필요한 설정 예시를
기록합니다.

공유하지 않을 것

비밀번호·API 키·개인정보를 커밋하지 않습니다.
유출된 키는 삭제만 하지 말고 폐기·재발급합니다.

이름 변경과 삭제도 커밋에 기록됩니다

실행 예시

```
git mv index.html home.html
git status --short
# R  index.html -> home.html
git commit -m "refactor: rename page"

git rm home.html
git status --short
# D  home.html
git commit -m "chore: remove old page"
```

git mv

파일을 이동하고 관련 변경을 스테이징합니다. Git은 내용 유사성으로 이름 변경을 추정합니다.

git rm

추적 중인 파일을 작업 폴더에서 삭제하고 삭제를 스테이징합니다.

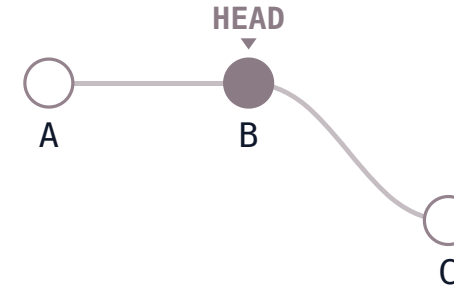
독립 예제

연습용 파일을 사용합니다. 이전 파일은 커밋 이력에서 읽을 수 있습니다.

switch: 같은 커밋에서 작업을 나누기

명령 · 클릭하면 해당 단계

```
01 git log --oneline -2
02 git switch -c feature/title
03 echo "<h1>새 동아리</h1>" > index.html
04 git add index.html
  git commit -m "feat: new title"
05 git switch main
```



main B

feature/title C

실행 결과 예시 · 출력 일부 생략

Switched to branch 'main'

05 HEAD가 main으로 돌아갑니다. clean 상태라면 작업 파일도 B의 내용으로 바뀝니다.

merge는 현재 브랜치에 다른 이력을 합칩니다

Git 개념 · 코드로 읽기

```
# feature/title의 변경을 main에 합치기
git switch main
git merge feature/title
```

```
# 현재 브랜치와 이력 보기
git branch --show-current
git log --graph --oneline --all
```

```
# 반대 방향은 다른 작업입니다
# git switch feature/title
# git merge main
```

먼저 현재 브랜치 결정

git switch main으로 이동한 뒤 병합하면 결과를 main에 반영합니다.

인자는 가져올 이력

git merge feature/title에서 feature/title은 합칠 대상입니다. 해당 브랜치 이름은 그대로 남습니다.

상황에 따라 결과가 달라짐

직선으로 이어지면 fast-forward가 가능하고, 갈라졌으면 병합 커밋이나 충돌 처리가 필요할 수 있습니다.

충돌 예제: 같은 줄을 서로 다르게 수정

실행 예시

```
git switch -c feature/name
echo "<h1>AI Club</h1>" > index.html
git add index.html
git commit -m "feat: AI title"

git switch main
echo "<h1>Web Club</h1>" > index.html
git add index.html
git commit -m "feat: Web title"
git merge feature/name
```

같은 출발점

초기 index.html은 `<h1>Club</h1>` 한 줄이고 이미 커밋된 상태입니다.

서로 다른 수정

feature/name은 AI Club, main은 Web Club으로 같은 줄을 바꿉니다.

merge 실행 결과

자동 병합이 멈추고 index.html에 충돌 표시가 생깁니다. 이 상태에서 내용을 편집합니다.

충돌 표시를 최종 코드로 바꾸기

명령 · 클릭하면 해당 단계

01 `git status --short`

02 `cat index.html`

03 `echo "<h1>AI & Web Club</h1>" > index.html`

04 `git add index.html`

05 `git commit -m "merge: combine titles"`

main
두 이력 병합

→

feature/name
이력 유지

작업 파일
최종 내용

←

병합 상태
완료

실행 결과 예시 · 출력 일부 생략

`[main e5f6a7b] merge: combine titles`

05 해결 결과를 병합 커밋으로 기록합니다. 진행 중 병합을 취소할 때는 git merge --abort입니다.

GitHub에 원격 저장소 만들기

01. 저장소 생성

GitHub에 로그인하고 **New repository**에서 `club-site`를 만듭니다. README를 포함합니다.

02. 공개 범위

Pages 선택 실습까지 진행한다면 **개인정보 없는 Public** 저장소를 사용합니다. 공개가 어려우면 Private으로 실습하고 배포는 생략합니다.

03. 로컬 복제

저장소의 **Code** 메뉴에서 URL을 복사합니다. 기존 club-site-local 폴더 밖에서 새 폴더로 clone합니다.

clone과 remote: 원격 저장소 연결

명령 · 클릭하면 해당 단계

01 `git clone https://github.com/USER/club-site.git`

02 `cd club-site`
`git remote -v`

03 `git status`

GitHub
main

→ 로컬 폴더
파일 복제 완료

origin
GitHub 주소

← 현재 브랜치
main · clean

실행 결과 예시 · 출력 일부 생략

On branch main
nothing to commit, working tree clean

03 복제한 폴더 안에서 상태를 봅니다. 아직 수정하지 않아 커밋할 변경이 없습니다.

원격 연결 오류를 메시지별로 구분하기

실행 예시

```
git remote -v
```

```
# 주소 또는 접근 권한 문제  
# Repository not found
```

```
# 인증 문제의 한 예  
# Permission denied (publickey)
```

```
# 브라우저에서 저장소 주소 열기
```

주소와 권한

Repository not found는 주소 오류뿐 아니라 비공개 저장소 접근 권한이 없을 때도 나타납니다.

접속 방식 살펴보기

publickey 오류는 접속 방식 살펴보기와 관련됩니다. 등록된 키와 사용 계정을 살펴봅니다.

로그인 계정과 권한

브라우저에서 저장소 주소 열기는 GitHub CLI 로그인 상태입니다. 모든 Git 전송 방식의 인증을 보장하지 않습니다.

commit은 로컬에, push는 원격에

명령 · 클릭하면 해당 단계

01 `echo "소개 문장" >> README.md`

02 `git add README.md`

03 `git commit -m "docs: add intro"`

04 `git push origin main`

작업 파일
clean

→

스테이징
HEAD와 동일

로컬 main
B

←

GitHub main
B

실행 결과 예시 · 출력 일부 생략

a1b2c3d..b2c3d4e main -> main

04 인증과 쓰기 권한이 있으면 원격 main도 B로 이동합니다. 원격이 앞서 있으면 거절될 수 있습니다.

pull로 GitHub의 최신 변경을 가져옵니다

기본 실습

```
# 내 수정은 먼저 커밋한 상태
# main에서 새 작업을 시작하기 전

git switch main
git status
# nothing to commit, working tree clean

git pull --ff-only
# GitHub의 새 커밋을 로컬에 반영
```

push와 방향이 반대

push는 내 커밋을 보내고, pull은 원격의 변경을 가져와 현재 브랜치에 반영합니다.

시작 전에 최신 상태로

동료가 작업했거나 웹에서 PR을 병합했다면 로컬 파일은 아직 예전 상태일 수 있습니다.

--ff-only로 단순 갱신

양쪽 이력이 갈라지면 중단합니다. 이때 명령을 반복하기보다 메시지를 읽고 감사와 함께 정리합니다.

push가 거절되면 오류 메시지부터 읽습니다

기본 실습

git push

출력 예시

rejected: non-fast-forward

지금 상태를 먼저 보기

git status

여기서 멈추고 원격 변경과 내 작업을

강사와 함께 살펴봅니다.

원격에 다른 변경이 있음

동료의 변경이 먼저 올라갔다면 내 이력이 최신 상태를 포함하지 않을 수 있습니다.

내 작업을 먼저 보존

수정한 파일이 남아 있는지 status로 봅니다. 오류 메시지를 함께 기록해 도움을 요청합니다.

덮어쓰기보다 내용 비교

강제 push로 해결하려 하지 않습니다. 처음에는 강사와 함께 변경을 합친 뒤 다시 보냅니다.

작업 브랜치를 원격에 게시하기

실행 예시

```
git switch main
git pull --ff-only
git switch -c docs/intro
echo "프로젝트 소개" >> README.md
git add README.md
git commit -m "docs: add project intro"
git push -u origin docs/intro
```

main에서 출발

clean main을 원격과 맞춘 후 작업 브랜치를 만듭니다.

브랜치 이름

docs/intro는 문서 소개 작업을 나타냅니다.
main의 내용은 병합 전까지 유지됩니다.

-u 옵션

원격 docs/intro와 추적 관계를 설정합니다. 이후 같은 브랜치에서는 git push를 쓸 수 있습니다.

git push -u origin feature/title을 나누어 읽기

Git 개념 · 코드로 읽기

```
git push -u origin feature/title
#      |  |      └ 보낼 로컬 브랜치
#      |  └      └ 원격 저장소의 별명
#      └      └ upstream 연결 설정

# 연결 상태 보기
git branch -vv

# 일반적인 설정에서는 이후 같은 브랜치에서
git push
# 인자를 생략해 연결된 원격 브랜치로 전송
```

push는 커밋을 전송

저장하거나 add만 한 변경은 전송되지 않습니다.
먼저 보낼 브랜치에 커밋이 있어야 합니다.

-u는 연결을 기억

성공적으로 게시한 브랜치에 upstream을
설정합니다. 이후 상태 비교와 기본 push·pull에
사용됩니다.

PR과 배포는 별도 흐름

push했다고 PR이 자동 생성되지는 않습니다. 배포
자동화가 설정되어 있다면 push가 배포를 시작할
수 있습니다.

Pull Request: 게시한 변경을 검토하고 병합

명령 · 클릭하면 해당 단계

01 `git push -u origin docs/intro`

02 GitHub: New pull request

03 GitHub: Files changed → Review

04 GitHub: Merge pull request

05 `git switch main`
`git pull --ff-only`

작업 브랜치
병합됨

→

Pull Request
Merged

main
새 내용

←

로컬 main
새 내용

실행 결과 예시 · 출력 일부 생략

Fast-forward

05 GitHub에서 병합해도 내 작업 폴더는 자동으로 바뀌지 않습니다. 로컬도 동기화합니다.

PR에서는 합칠 방향과 바뀐 파일을 읽습니다

기본 실습

```
base: main
compare: docs/intro
```

```
# 의미
# docs/intro의 변경을 main에 반영 요청
```

```
Files changed
- 이전 소개 문장
+ 수정한 소개 문장
```

```
Merge pull request
# 검토를 마친 뒤 권한이 있는 사람이 병합
```

main은 결과를 받을 곳

base는 변경을 합칠 브랜치입니다. compare는 내가 작업해서 올린 브랜치입니다.

Files changed로 내용 보기

의도한 파일과 줄만 바뀌었는지 읽습니다. 수정이 더 필요하면 같은 브랜치에서 커밋하고 push합니다.

병합 후 로컬도 갱신

GitHub의 main에 반영되어도 내 컴퓨터 파일은 즉시 바뀌지 않습니다. main으로 이동하고 pull합니다.

열린 PR에 수정 커밋 추가하기

실행 예시

```
git switch docs/intro
echo "실행: index.html 열기" >> README.md
git add README.md
git commit -m "docs: add usage"
git push
```

```
# 브라우저에서 기존 PR 열기
# Files changed에서 추가 변경 읽기
```

같은 작업 브랜치

PR의 head 브랜치에 새 커밋을 push하면 열린 PR에도 추가 변경이 반영됩니다.

리뷰 피드백 반영

새 PR을 계속 만들기보다 같은 PR에서 수정 이유와 결과를 이어 설명할 수 있습니다.

다시 읽을 변경

추가 commit의 내용이 의도와 같은지 diff를 읽고 리뷰를 이어갑니다.

Issue에는 할 일을, PR에는 바꾼 내용을 적습니다

기본 실습

Issue 제목

README에 실행 방법 추가

Issue 내용

index.html을 여는 방법을 설명해주세요.

PR 제목

docs: add run instructions

PR 내용

README에 브라우저 실행 방법을 추가했습니다.

관련 Issue: #12

Issue는 작업 전

문제나 해야 할 일을 설명합니다. GitHub 저장소의 Issues 화면에서 작성합니다.

PR은 변경 후

어떤 파일을 왜 바꿨는지 적습니다. 코드를 올린 뒤 Pull requests 화면에서 작성합니다.

번호로 맥락 연결

실제 Issue 번호를 적으면 변경의 목적을 찾아가기 쉽습니다. 여기서 #12는 설명용 번호입니다.

Issue는 할 일, PR은 변경 검토입니다

Issue

할 일 / 버그 / 제안

무슨 일을 할지 이야기

“README에 실행 방법 추가”처럼 실행 결과를 적습니다.

Pull Request

변경 비교

리뷰 후 병합

완료한 변경을 반영 요청

같은 저장소의 브랜치끼리도 PR을 만듭니다. PR을 열면 자동 병합되는 것은 아닙니다.

GitHub 저장소와 배포된 웹사이트는 다릅니다

Repository

코드 · 이력

소스와 변경 기록을 공유

github.com/USERNAME/club-site

GitHub Pages

웹페이지

정적 파일을 웹으로 제공

USERNAME.github.io/club-site/
별도 설정이 필요합니다.

배포할 index.html 작성

실습 코드

```
<!doctype html>
<html lang="ko">
<head>
  <meta charset="utf-8">
  <title>나의 첫 프로젝트</title>
</head>
<body>
  <h1>초급프로젝트 2주차</h1>
  <p>서버와 GitHub를 배웠습니다.</p>
</body>
</html>
```

파일 만들기

저장소 최상위에 index.html로 저장합니다.
대소문자도 확인합니다.

협업 과정 재사용

새 브랜치에서 파일을 commit·push하고 PR로
main에 병합합니다. 웹 편집도 가능합니다.

실행 결과

GitHub의 main 파일 목록에서 index.html이
보입니다. 실습용 파일도 함께 제공됩니다.

GitHub Pages 배포 설정

01. 배포 설정

저장소 **Settings** → **Pages** → **Build and deployment**

Source: **Deploy from a branch**를 선택합니다.

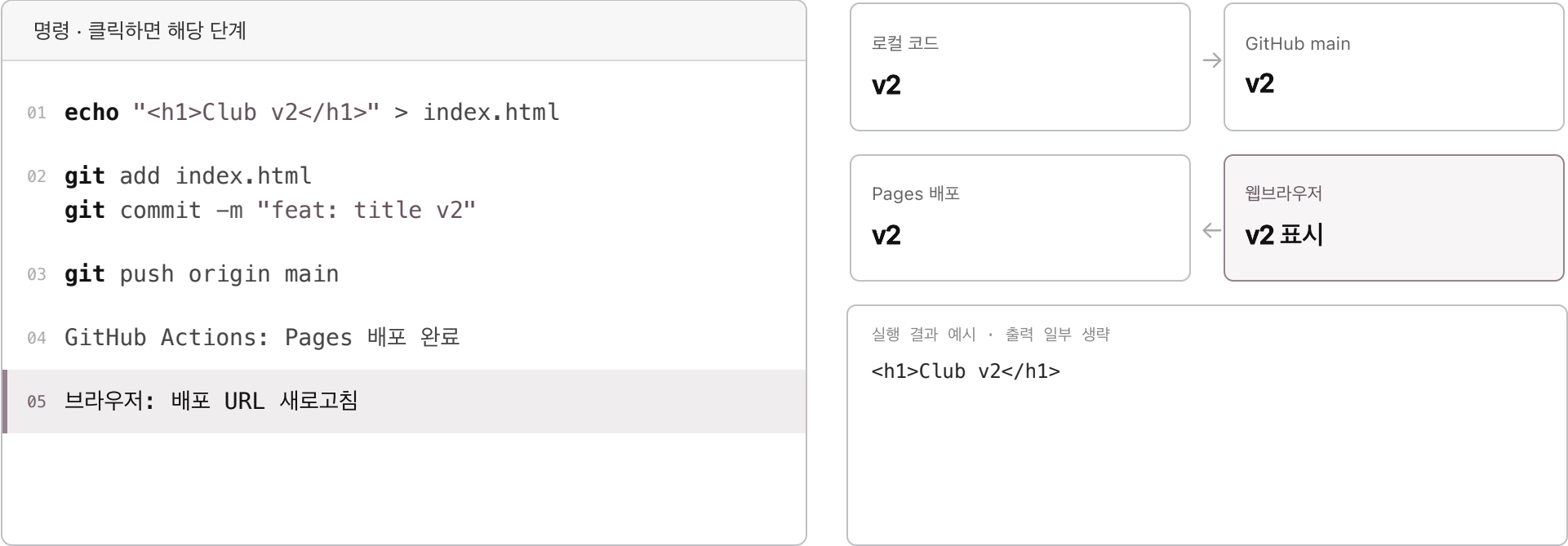
02. 배포할 위치

Branch: **main**, Folder: **/ (root)**를 선택하고 Save합니다. 저장소 최상위의 index.html을 제공합니다.

03. 배포 URL

배포 완료 후 Pages에 표시된 URL을 엽니다. 바로 반영되지 않으면 **Actions의 배포 상태**를 확인합니다.

코드 변경이 배포된 페이지에 반영되는 과정



05 새 배포가 적용된 페이지를 요청합니다. 캐시 상황에 따라 반영이 늦어질 수 있습니다.

참고자료와 더 읽을 문서

수업 참고자료

02. 프론트엔드서버.pdf · 03. Git and GitHub.pdf

예시Template의 한국어 강의 · Codex Workshop 디자인 반영

[MDN — What is a web server?](#)

[Pro Git — Recording Changes](#)

[GitHub — Hello World](#)

[GitHub — What is GitHub Pages?](#)

[GitHub — Pages 배포 소스 설정](#)